YZG102 Lab 6

İşaretçiler, Referanslar ve Dinamik Bellek

Yeşim Yiğitbaşı

10.04.2026

Görev 1

Bellek Adresleri ve İşaretçi Temelleri

Beklenen Çıktı

```
x = 42        &x = 0x61FF0C
*ptr = 42     ptr = 0x61FF0C
*ptr degistirildikten sonra x = 99
sizeof(int) = 4    sizeof(int*) = 8
```

1. int, double ve char tipinde birer değişken tanımlayın.

2. Her değişken için aynı tipte bir işaretçi (pointer) tanımlayın ve adreslerini atayın.

3. Her işaretçi üzerinden * ile değere erişin ve ekrana yazdırın.

4. * operatörü ile her değişkenin değerini değiştirin ve değişimin orijinal değişkene yansıdığını ekrana yazdırarak gösterin.

5. Her değişkenin ve her işaretçinin sizeof() değerini ekrana yazdırın.

Soru 1: int* ptr ile int ptr arasındaki fark nedir?

Soru 2: Bir işaretçi değişkeninin kendisi kaç byte yer kaplar? Neden?

Soru 1: int* ptr ile int ptr arasındaki fark nedir?

int ptr; sıradan bir tam sayı değişkenidir, bir değer saklar. int* ptr; ise bir işaretçi değişkenidir ve bir int değişkeninin bellek adresini saklar. ptr = 5 gibi bir atama ile değer tutulurken, ptr = &x şeklinde adres tutulur. *ptr ifadesi ile işaretçinin gösterdiği değere ulaşılır.

Soru 2: Bir işaretçi değişkeninin kendisi kaç byte yer kaplar? Neden?

64-bit sistemlerde bir işaretçi değişkeni her zaman 8 byte (64 bit) yer kaplar. 32-bit sistemlerde ise 4 byte olur. Bunun nedeni, işaretçinin bir bellek adresi tutmasıdır. Bellek adresi, kullanılan sistemin adres uzayı genişliğine bağlıdır. int* ile double* ile char* hepsinin boyutu aynıdır çünkü hepsi adres tutar, veri tipi işaretçinin boyutunu değil gösterdiği verinin boyutunu belirler.

Görev 2A

İşaretçi Aritmetiği ve Dizi İlişkisi

```
int arr[5] = {10, 20, 30, 40, 50};
int* ptr = arr;

// Aşağıdaki ifadelerin sonuçlarını sağ tarafa yazın:
cout << arr[0];         // Sonuç: ______
cout << *ptr;           // Sonuç: ______
cout << *(ptr + 2);     // Sonuç: ______
cout << arr[3];         // Sonuç: ______
cout << *(arr + 4);     // Sonuç: ______
ptr++;
cout << *ptr;           // Sonuç: ______
```

Bir dizi adı, o dizinin ilk elemanının adresidir. Bu sayede arr[i] ile *(arr+i) tamamen aynı anlama gelir.

Görev 2B

İşaretçi Aritmetiği ve Dizi İlişkisi

5 elemanlı int bir dizi tanımlayın ve kullanıcıdan değer okuyun.

Diziyi hem indeks hem de işaretçi aritmetiği ile yazdırın.

Her elemanın bellek adresini de gösterin.

Soru 1: ptr++ işlemi adresi kaç byte artırır? Neden?

Soru 1: ptr++ işlemi adresi kaç byte artırır? Neden?

ptr bir int* işaretçisi olduğu için ptr++ işlemi adresi 4 byte artırır (int veri tipi 4 byte yer kaplar). C++ işaretçi aritmetiğinde artış miktarı her zaman gösterilen veri tipinin boyutu kadardır. Eğer ptr bir double* olsaydı 8 byte, char* olsaydı 1 byte artardı. Bu sayede ptr++ ifadesi her zaman dizinin bir sonraki elemanını gösterir.

Görev 3A

Referans ile Fonksiyon Parametreleri

ikiyeKatla Fonksiyonu

Yandaki üç fonksiyonu tamamlayın ve main() içinde test edin.

Hangi versiyonun orijinal değişkeni değiştirdiğini gözlemleyin.

```
// Versiyon 1: Değer ile (by value)
void ikiyeKatlaV(int x) {
    x = x * 2;
}

// Versiyon 2: Referans ile (by reference)
void ikiyeKatlaR(int& x) {
    ______ = ______ * 2;
}


// Versiyon 3: İşaretçi ile (by pointer)
void ikiyeKatlaP(int* x) {
    ______ = ______ * 2;
}
```

Görev 3B

Referans ile Fonksiyon Parametreleri

minMaxBul Fonksiyonu

Bir dizi alan, minimum ve maksimum değerleri işaretçi parametresi ile döndüren fonksiyon yazın.

Soru 1: void ikiyeKatlaV() neden orijinal değişkeni değiştiremiyor?

Soru 2: Referans (&) ile işaretçi (*) parametre geçirmenin pratik farkı nedir?

Soru 1: void ikiyeKatlaV() neden orijinal değişkeni değiştiremiyor?

Değer ile (by value) parametre geçirildiğinde, fonksiyon orijinal değişkenin bir kopyasını alır. Fonksiyon içinde yapılan tüm değişiklikler bu kopya üzerinde gerçekleşir. Fonksiyon sona erdiğinde kopya bellekten silinir ve orijinal değişken hiç dokunulmamış olarak kalır. Orijinali değiştirmek için ya referans (&) ya da işaretçi (*) kullanılmalıdır.

Soru 2: Referans (&) ile işaretçi (*) parametre geçirmenin pratik farkı nedir?

Referans ile geçirmede fonksiyon çağrısı sade görünür: ikiyeKatlaR(a). İşaretçi ile geçirmede çağıran & işareti kullanmalıdır: ikiyeKatlaP(&a). Referans null olamaz ve yeniden atanamaz, bu yüzden daha güvenlidir. İşaretçi null olabilir ve null kontrolü yapılabilir, bu yüzden opsiyonel parametreler için tercih edilir. Modern C++'da genellikle referans tercih edilir.

Görev 4A

Dinamik Bellek: new ve delete

Heap bellekte new ile alan ayırmayı ve delete ile serbest bırakmayı öğrenmek.

```
// Tek değişken
int* p = ______;          // heap'te int için yer aç
*p = 75;
cout << *p << endl;       // 75
______ p;                 // belleği serbest bırak
p = ______;               // güvenlik için

// Dinamik dizi (n=5)
int n = 5;
int* arr = ______ int[n];
for (int i = 0; i < n; i++)
    arr[i] = i * 10;
______ arr;               // DİKKAT: tek değerden farklı!
arr = nullptr;
```

Görev 4B

Dinamik Bellek: new ve delete

Kullanıcıdan bir n değeri alın. n elemanlı dinamik double dizi oluşturun.
Değerleri okuyun, toplamı ve ortalamasını hesaplayın.
Belleği doğru şekilde serbest bırakın.

Soru 1: Stack bellek ile Heap bellek arasındaki en önemli iki fark:

Soru 2: delete p; ile delete[] p; arasındaki fark nedir?

Soru 1: Stack bellek ile Heap bellek arasındaki en önemli iki fark nelerdir?

1. Yönetim: Stack bellek otomatik yönetilir, değişken kapsam dışına çıkınca otomatik silinir. Heap bellek ise programcı tarafından new ile ayrılır ve delete ile serbest bırakılmalıdır. 2. Boyut: Stack belleği sınırlıdır (genellikle 1-8 MB). Heap belleği çok daha büyüktür, sistemin RAM'i kadar kullanılabilir. Bu yüzden büyük veya boyutu çalışma zamanında belirlenen veriler için heap tercih edilir.

Soru 2: delete p; ile delete[] p; arasındaki fark nedir?

delete p; tek bir nesne için ayrılan belleği serbest bırakır (new int veya new double gibi). delete[] p; ise bir dizi için ayrılan belleği serbest bırakır (new int[n] gibi). Yanlış kullanılırsa tanımsız davranış (undefined behavior) oluşur. Yani new[] ile ayrılmış bellek için delete (köşeli parantez olmadan) kullanılırsa, yalnızca dizinin ilk elemanının yıkıcısı çalışır ve geri kalan bellek sızdırılır. Bu tür hatalar programın çökmesine veya bellek bozulmasına yol açabilir.

Görev 5

Dinamik Dizi ile Struct Uygulaması

Yapılacaklar

1. Kullanıcıdan kaç ürün girileceğini sorun (n).
2. n elemanlı dinamik Urun dizisi oluşturun:
   Urun* envanter = new Urun[n];
3. Döngü ile tüm ürünlerin bilgilerini okuyun.
4. Toplam stok değerini hesaplayın:
   Σ(stok × fiyat)
5. En pahalı ürünü bulun ve bilgilerini yazdırın.
6. delete[] ile belleği serbest bırakın.

```
struct Urun {
    string ad;
    int    stok;
    double fiyat;
};
```

Soru 1: new Urun[n] ile new int[n] arasındaki fark:

Soru 2: delete[] unutulursa ne olur?

Soru 3: İşaretçi ile struct üyesine erişmek için hangi operatör kullanılır?

Soru 1: new Urun[n] ile new int[n] arasındaki fark:
new int[n] ilkel (primitive) bir veri tipi için dizi ayırır. new Urun[n] ise kullanıcı tanımlı bir struct için dizi ayırır. Kullanım açısından fark yoktur; her ikisinde de delete[] ile serbest bırakılmalı ve elemanlarına indeks ile erişilmelidir. Ancak Urun struct içerdiğinden her elemana nokta operatörü ile alt alanlara erişilir: envanter[i].ad gibi.

Soru 2: delete[] unutulursa ne olur?
Kısa vadede program çalışmaya devam eder ve kullanıcı bir sorun fark etmez. Uzun vadede bu bellek program sonlanana kadar işletim sistemine iade edilmez, yani bellek sızıntısı (memory leak) oluşur. Uzun süre çalışan programlarda (sunucu uygulamaları gibi) bu sızıntılar birikerek tüm belleğin tükenmesine ve programın çökmesine yol açar.

Soru 3: İşaretçi ile struct üyesine erişmek için hangi operatör kullanılır?
Arrow operatörü (->) kullanılır. Örneğin: Urun* ptr = &envanter[0]; ptr->ad şeklinde erişilir. Bu ptr->ad ile (*ptr).ad ifadeleri tamamen aynı anlama gelir. Dizi elemanları için ise nokta operatörü yeterlidir çünkü envanter[i] zaten bir Urun nesnesidir, işaretçi değildir.

Bonus

Sarkan İşaretçi (Dangling Pointer) Analizi

Aşağıdaki kodu inceleyin ve her satırın ne yaptığını, varsa hata veya tehlikeyi açıklayın.

```
int* p = new int(42);
int* q = p;
delete p;
p = nullptr;
cout << *q;        // (1) Bu satır ne yapar?
delete q;          // (2) Bu satır ne yapar?
```

Bonus

Sarkan İşaretçi (Dangling

Aşağıdaki kodu inceleyin ve
tehlikeyi açıklayın.

```
int* p = new int(42);
int* q = p;
delete p;
p = nullptr;
cout << *q;        // (1
delete q;          // (2
```

(1) Analiz: *q tanımsız davranış (undefined behavior) üretir. p ile ayrılan bellek delete p ile serbest bırakıldı. Ancak q hâlâ o adresi tutuyor. Serbest bırakılmış bellekten okumak C++'da tanımsız davranıştır: program çalışmaya devam edebilir (eski değeri döndürebilir), hatalı değer döndürebilir veya çökebilir.

(2) Analiz: delete q; çift silme (double delete) hatasıdır. Aynı bellek adresi zaten delete p; ile serbest bırakıldı. Aynı bloğu iki kez silmek tanımsız davranıştır ve büyük ihtimalle programın çökmesine neden olur.

(3) Bu problemi nasıl düzeltirdiniz? delete p; ardından hem p = nullptr hem de q = nullptr yapılmalıdır. Bu sayede q ile *q erişimi nullptr dereference hatası verir (kontrol edilebilir), ama en azından tanımsız davranış olmaz. Daha iyi çözüm: aynı bellek için iki ham işaretçi kullanmaktan kaçınmak, modern C++'da std::shared_ptr kullanmaktır.